VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

★ ☆ ★

CS486 COURSE REPORT – PHASE 2

Campus Space Management System

Members of group 05:	MSSV:
Phạm Hữu Nam	24125015
Cao Quảng Hưng	24125010
Nguyễn Hữu Phước	24125018
Trần Đình Quốc Thắng	24125079

TP. Hồ Chí Minh, 08/2026

Mục lục

1 LLM Usage & Agent Improvement Process

1.1 LLM Models Used

- **Big Pickle:** Evaluated via the OpenCode Zen platform, this model is specifically optimized for automated coding agents. Featuring an extensive context window of approximately 200k tokens, it serves as an ideal tool for rapid initial prototyping. However, its primary limitation lies in the absence of advanced reasoning features.
- **DeepSeek V4 Flash Free:** Accessed through the OpenCode platform, this model was deployed to facilitate technical drafting, content refinement, and structural editing, such as database design documentation. It demonstrates high efficiency in generating structured explanations and concise academic prose, though it necessitates human oversight to ensure rigorous factual accuracy and alignment with project specifications.

1.2 Evaluation and Improvement Cycle

Explain how agent output was evaluated after each Phase 2 task. Summarize the problems found, the evidence used to identify them, and the corresponding changes to prompts, `AGENTS.md`, skills, validation steps, or review checklists.

Task	Evaluation evidence	Issue found	Improvement made
08–16	To be completed	To be completed	To be completed

Bång 1: Phase 2 agent evaluation and improvement log

2 Requirement Change Summary

This section summarizes the results of `08-requirement-change-analysis`. It should distinguish new Phase 2 requirements from rules retained from Phase 1.

2.1 Changes from Phase 1

- Define maintenance impact levels, including out-of-service and advisory maintenance.
- Explain how advisory maintenance affects bookings and acknowledgements.
- Describe the new instant-booking operating condition.
- Describe the expected behavior under concurrent access.

2.2 Affected Data and Business Rules

Identify the affected entities, relationships, attributes, and constraints. Provide traceability from each changed requirement to the corresponding design and implementation artifact.

3 Updated Design

This section presents the results of `09-updated-erd-and-logical-design`.

Change ID	Affected data	Business rule	Implemented in
To be completed	To be completed	To be completed	To be completed

Bång 2: Traceability of Phase 2 requirement changes

3.1 Updated Entity–Relationship Diagram

Describe the entities and relationships added or changed for Phase 2, including maintenance impact levels and advisory acknowledgements.

3.2 Updated Relational Schema

List all new and changed tables, primary keys, foreign keys, uniqueness rules, check constraints, and other integrity controls.

Table	Change type	Key or constraint changes	Reason
To be completed	To be completed	To be completed	To be completed

Bång 3: Phase 2 relational schema changes

3.3 Design Justification

Justify the main design decisions and explain how they preserve data integrity, support concurrency, and remain compatible with existing Phase 1 data.

4 Schema Migration

Phase 2 extends the frozen Phase 1 schema with maintenance impact levels, advisory acknowledgements, and data-driven instant booking. Since the database already contained booking and maintenance data, the upgrade was implemented as a data-preserving T-SQL delta in `10-schema-migration.sql`. The script upgrades the Task 05 baseline, `05-db-definition.sql`, to the approved Task 09 design. The companion script `10-schema-migration-rollback.sql` reverses the upgrade when required.

4.1 Migration Approach

The migration is primarily additive: it introduces new columns, tables, indexes, and triggers without rebuilding or renaming any Phase 1 table. The only removed element is `spaces.usage_policy`, an unused free-text column superseded by the Task 09 reference-table design. No rule, index, or trigger depended on this column.

The complete schema delta is summarized in Table 4.

Schema component	Phase 2 implementation
Maintenance impact	Add `maintenance.impact_level` as `NOT NULL`, defaulting to `'out-of-service'`. A check constraint permits only `out-of-service` and `advisory`.
Impact history	Add `maintenance_impact_history` to record every maintenance impact-level change.
Advisory acknowledgement	Add `booking_advisory_acknowledgement`. A composite unique constraint permits one acknowledgement per booking–advisory pair.
Instant-booking policy	Add `space_type_allowed_purpose` and remove the superseded `spaces.usage_policy` column.
Trigger logic	Restrict booking blocks to out-of-service maintenance; record impact history; validate acknowledgement ownership; maintain modification timestamps; and recompute space status.
Configuration data	Seed one reserved system approver and 18 allowed-purpose pairs for the four approved space types.

Bång 4: Phase 2 schema migration delta

The revised booking and maintenance triggers block overlaps only when a space has out-of-service maintenance; advisory maintenance produces a warning but does not make the space unavailable. Additional triggers record impact-level changes, verify that acknowledgements are submitted by the booking owner, maintain last-modified timestamps, and recompute `spaces.current_status`. Status is derived using the approved priority order, avoiding duplicated state.

The migration is both transactional and repeatable. All operations execute in a single transaction, so a failed or interrupted run leaves the database unchanged. Existence checks guard object creation and seed insertion, allowing the script to be rerun without creating duplicate objects or rows. The rollback script restores the four Phase 1 trigger definitions and removes every column, table, index, trigger, and seed row introduced by the migration, returning the schema to the Task 05 baseline.

4.2 Data Preservation and Backfill

No existing booking or maintenance row is deleted or assigned a new identity. The only backfill sets the new maintenance impact level to `'out-of-service'`. This preserves Phase 1 behavior because every existing active maintenance record continues to block overlapping bookings.

After the schema changes, `spaces.current_status` is recomputed using the priority order: retired, temporarily closed, out-of-service maintenance, live session, and available. Existing manual `retired` and `temporarily_closed` overrides are preserved.

The migration also inserts the reserved system approver used to identify derived instant approvals and the 18 `(space_type, purpose)` pairs that define instant-booking eligibility for the four approved space types. Guarded inserts ensure that each row is created only once and do not disturb existing identity values.

4.3 Verification

Verification was performed on a scratch SQL Server 2022 database. The Task 05 DDL and sample data were loaded first, followed by the migration, a second run of the same migration, and the rollback. Validation queries confirmed that the new schema objects had the approved definitions, every maintenance row had an impact level, and the system approver and all 18 allowed-purpose pairs existed exactly once. The second migration run completed without errors or duplicates, confirming repeatability.

Transactional smoke tests covered the main behavioral rules. Advisory maintenance allowed overlapping bookings, whereas out-of-service maintenance continued to block them. Duplicate acknowledgement of the same advisory was rejected, as was approval without the required acknowledgement. Each test was rolled back after execution. Finally, the rollback script restored the baseline row counts exactly, confirming that the migration and reversal caused no data loss.

5 Concurrency Design & Implementation

Phase 2 introduces a high-concurrency operating condition: at the start of a semester, many users may request popular spaces for overlapping periods within a short interval. Eligible requests are confirmed immediately through instant booking, whereas other requests follow the staff-approval workflow. Since both pathways can confirm the same space–time interval concurrently, the database must enforce NR 6: *two approved bookings must never use the same space during overlapping periods, regardless of how they were created.*

Task 08 identified five concurrency conflicts (K1–K5) that can violate this invariant or expose stale availability. The solution was specified in `11-concurrency-design`, implemented in `12-concurrency-implementation.sql`, and verified through deterministic two-session tests in `13-concurrency-tests`. It serializes the four write paths that can change the confirmed state of a space–time interval: instant submission, staff approval, maintenance escalation or downgrade, and maintenance-ticket creation. Each path uses the same logical resource for a space and rechecks the relevant invariants immediately before writing.

5.1 Identified Concurrency Conflicts

The primary conflict is the K1 *read–check–write race*. Two concurrent requests for the same space and period can both observe the interval as available and then confirm their bookings before either transaction sees the other's result. Table 5 illustrates this race for two instant-booking sessions targeting space S_1 from 10:00 to 12:00.

Step	Session A: instant submission	Session B: instant submission
1	Check S_1: no confirmed overlap or blocking maintenance.	
2		Check S_1: no confirmed overlap or blocking maintenance.
3	Insert a pending booking and its automatic approval (`approver_id = -1`).	
4		Insert a pending booking and its automatic approval; BR1 and BR4 checks see no committed competitor.
5	Commit.	
6		Commit.

Bång 5: Concurrent instant-booking race for the same space and period

At step 4, session A's write remains uncommitted and is therefore not visible to session B under the default READ COMMITTED isolation level. Both transactions pass the availability rule (BR1) and maintenance rule (BR4), then commit overlapping approved bookings on S_1, violating BR1 and NR6.

The interval between the availability check and the confirmation write is the *race window*. Because no lock spans this interval, neither existing backstop fully closes it. The trigger **trg_bookings_prevent_overlap** cannot validate against a competing row that is not yet visible, while the filtered unique index **uq_bookings_active_overlap** prevents only identical start times. It does not reject intersecting intervals with different start times, such as 10:00–12:00 and 11:00–13:00.

The same check–write race occurs across other booking and maintenance paths. K2 covers instant submission racing with staff approval. K3 occurs when a maintenance record is escalated to **out-of-service** during an in-flight confirmation, while K5 covers insertion of a new maintenance ticket whose default impact is **out-of-service**. K4 is a read-side conflict in which room-finder availability becomes stale before the user acts. Table 6 summarizes the complete conflict inventory.

ID	Concurrent scenario	Affected rules	Path
K1	Two requests read the same interval as free and both confirm.	BR1, NR6	Booking insertion
K2	Instant submission races with staff approval.	BR1, NR6	Cross-pathway confirmation
K3	Maintenance is escalated while a booking confirmation is in flight.	BR4, NR4	Maintenance update
K4	Room-finder availability becomes stale before confirmation.	BR1, NR4, NR6	Read-only availability
K5	An out-of-service ticket is inserted during confirmation.	BR4, NR6	Maintenance insertion

Bång 6: Concurrency conflicts identified in the Task 08 analysis

K1, K2, K3, and K5 share the same root cause: concurrent transactions perform a check and a conflicting write on the same space–time interval without a common serialization point. K4 cannot by itself corrupt the database, but its stale result must not be trusted during

confirmation. The implemented control therefore serializes the check–recheck–write sequence per space and validates availability again inside the protected transaction. Consequently, two conflicting confirmations cannot both commit.

5.2 Cause and Chosen Control

5.2.1 Cause

The race occurs under the default READ COMMITTED isolation level because a transaction cannot observe another transaction's uncommitted rows. Two writers can therefore validate the same availability state before either commits. The existing triggers execute within their writer's transaction and cannot validate against an invisible competitor, while the filtered unique index rejects only identical start times. Since no common serialization point spans the check–write interval, all confirmation-related write paths require one.

5.2.2 Application-Lock Protocol

The implementation uses a transaction-owned SQL Server application lock. After starting a transaction, each write procedure calls **sys.sp_getapplock** with the following configuration:

- Resource: **space_booking:<space_id>**;
- Mode: **Exclusive**;
- Owner: **Transaction**;
- Timeout: 5 seconds.

The lock is released automatically when the transaction commits or rolls back. The procedures **usp_booking_instant_submit**, **usp_booking_approve**, **usp_maintenance_set_impact_level**, and **usp_maintenance_report** request the same resource for a given space. This serializes instant approvals, staff approvals, and booking–maintenance interactions through one protocol. Maintenance reporting acquires the lock only when the new ticket begins as **out-of-service**; an advisory ticket does not block the interval and instead creates an acknowledgement requirement.

5.2.3 Invariant Revalidation and Failure Handling

The lock protects the critical section, but correctness depends on revalidation inside it. Immediately before writing, each procedure rereads its target data and checks confirmed-booking overlap (BR1), out-of-service maintenance overlap (BR4), space status (BR2), capacity (BR3), and acknowledgement completeness (NR2). A failed check rolls back the transaction and returns a deterministic result code. Existing triggers and the filtered unique index remain as defense-in-depth controls.

Lock-acquisition failures are returned to the caller rather than silently retried. A timeout returns `51005`, a cancelled request returns `51006`, and both are retryable. A deadlock victim returns `51007`; in this case, the caller must restart the entire transaction.

5.2.4 Design Rationale and Trade-offs

The application lock is valid under READ COMMITTED and requires no schema change. In contrast, SERIALIZABLE key-range locking or UPDLOCK / HOLDLOCK range reads are reliable only when every writer locks the same predicate range. The four write paths use

different interval operations, making this approach plan-sensitive and vulnerable to a range-boundary error that could reopen K3 or K5. Optimistic concurrency would require version columns excluded from the approved design, while triggers alone cannot serialize concurrent writers under READ COMMITTED.

The lock resource is keyed by space rather than by space and day. A daily key would require a multi-day booking to acquire several locks in a fixed order, creating additional lock-ordering and cross-midnight risks. The per-space key avoids these cases but serializes all writers for a popular space. This cost is acceptable for the expected workload: Task 14 generates more than 100,000 bookings across three academic years, corresponding to tens of writes per day for the busiest space, and each critical section completes well within the five-second timeout. If production monitoring later identifies material contention, a per-day resource key remains the documented tuning option.

5.3 Concurrent Test Results

To evaluate the effectiveness and reliability of the Task 12 concurrency control mechanisms, a comprehensive **Comparison Suite** was developed in `outputs/13-concurrency-tests-G05`. Each conflict scenario is delivered as a pair:

- **Baseline (Uncontrolled - Raw DML):** Direct T-SQL execution (`INSERT`, `UPDATE`) without application-level locks, operating under `READ_COMMITTED_SNAPSHOT ON` (RCSI) mode. Under RCSI, Phase 1 triggers read committed row versions from `tempdb` without taking Shared Locks. When two transactions execute concurrently before either commits, Phase 1 triggers are bypassed, committing overlapping rows to disk ($Q_{BR1} \geq 1$), or throwing unhandled engine exceptions (`Msg 50000`) when a competitor commits first.

- **Controlled (Task 12 Stored Procedures):** All write operations route strictly through Task 12 entry points (`usp_booking_instant_submit`, `usp_booking_approve`, `usp_maintenance_report`, `usp_maintenance_set_impact_level`). Each procedure acquires an Exclusive SQL Server Application Lock (`sys.sp_getapplock` on `space_booking:<space_id>`) with a 5-second timeout, ensuring 100% data invariant preservation ($Q_{BR1} = 0$, $Q_{NR6} = 0$) and structured business result codes (`51001`–`51009`).

5.3.1 Test Architecture and Methodology

- **Dual-Session Parallel Execution:** An automated test runner orchestrates two independent database client sessions (Session A and Session B) concurrently in parallel within the same millisecond to force genuine lock contention and induce race condition windows under active workload.

- **Isolated Test Environment:** An automated setup script seeds an isolated test workspace featuring dedicated departments, accounts, and space instances with booking time windows scheduled far into the future ($\geq$ +600 days). This guarantees zero collision or data corruption with existing production sample data.

- **Idempotent Lifecycle Management:** An automated teardown script executes prior to and following every test scenario to perform complete, foreign-key-safe data cleanup, ensuring full test repeatability and isolation across test runs.

5.3.2 Evaluation Criteria

- **Baseline Verification (Vulnerability Demonstration):** A Baseline scenario passes verification by empirically exposing the systemic flaws of raw DML under concur-

rency—either through trigger bypass resulting in committed overlapping data corruption ($Q_{BR1} \geq 1$ or $Q_{NR6} \geq 1$), or via unhandled database engine exceptions (`Msg 50000` / `Msg 3609`) that abort client transactions.

- **Controlled Verification (Application-Level Control):** A Controlled scenario passes verification when it satisfies three mandatory guarantees:

  1. *Data Integrity Preservation:* Zero overlapping confirmed bookings ($Q_{BR1} = 0$) and zero out-of-service maintenance conflicts ($Q_{NR6} = 0$).
  2. *Application Serialization:* The winning transaction receives `rc = 0`, while the losing transaction receives an explicit structured business status code (`51003`, `51002`, `51005`).
  3. *Zero Engine Crashes:* 0% unhandled database engine exceptions (`Msg 50000`), allowing client applications to handle responses gracefully.

5.3.3 Scenario Analysis and Results

The comparison suite evaluates nine distinct concurrency test scenarios representing the five conflict families (K1–K5) and additional operational edge cases. Rather than embedding verbose T-SQL code listings into the document body, each scenario is abstracted below by its core objective, observed baseline vulnerability, and controlled resolution:

1. **Scenario 1: Concurrent Instant Submissions (`b01` vs `c01`)**

   - *Objective:* Verify system behavior when two users simultaneously submit instant bookings for the exact same space and overlapping time window.
   - *Baseline Result:* Under RCSI isolation, Phase 1 triggers read committed row versions from `tempdb` without taking Shared Locks. Both concurrent `INSERT` statements execute without blocking, bypassing the availability check and committing two overlapping confirmed bookings to disk ($Q_{BR1} = 1$).
   - *Controlled Result:* The Exclusive Application Lock on `space_booking:<space_id>` serializes execution. Session A acquires the lock first and commits (`rc = 0`). Session B waits, rechecks availability inside the lock, detects Session A's confirmed row, and rejects the submission with structured business code `51003` (BR1 Overlap Conflict), preserving $Q_{BR1} = 0$.

2. **Scenario 2: Instant Submission vs. Staff Approval (`b02` vs `c02`)**

   - *Objective:* Test cross-pathway concurrency when an instant booking submission races with a staff member approving an existing pending booking for the same interval.
   - *Baseline Result:* Without application locks, a raw `INSERT` for an instant booking commits concurrently alongside a staff approval transaction, producing overlapping approved bookings on disk ($Q_{BR1} = 1$).
   - *Controlled Result:* Both pathways acquire the same per-space application lock resource. Whichever procedure executes first acquires the lock and confirms its booking (`rc = 0`); the subsequent procedure revalidates availability within the lock and returns `51003`, maintaining $Q_{BR1} = 0$.

3. **Scenario 3: Maintenance Escalation vs. In-flight Confirmation (`b03` vs `c03`/`c03b`)**

   - *Objective:* Evaluate protection when a staff member escalates an existing advisory maintenance record to **out-of-service** during an active booking confirmation.

- *Baseline Result:* Raw DML execution causes the competing booking submission to hit the Phase 1 trigger after the escalation commits, throwing an unhandled engine exception (`Msg 50000`) that aborts the client batch.
- *Controlled Result:* `usp_maintenance_set_impact_level` acquires the per-space lock before altering the impact status. The escalation completes (`rc = 0`); any concurrent or subsequent booking attempt over the out-of-service window is rejected cleanly with `51002` (BR4 Out-of-Service Conflict), ensuring $Q_{NR6} = 0$.

4. **Scenario 4: Lock Timeout and Client Retry (`b05` vs `c05`)**
   - *Objective:* Validate system resilience and timeout contracts when a transaction holds a space resource lock during an extended operation.
   - *Baseline Result:* Raw row-level updates block competing transactions indefinitely (blocking observed for 18+ seconds), exposing clients to unbounded connection hangs.
   - *Controlled Result:* `usp_maintenance_report` enforces a strict 5-second lock timeout. The competing session times out after 5 seconds, returning retryable error code `51005`. A subsequent retry after the first transaction releases the lock succeeds with `rc = 0`.
5. **Scenario 5: Out-of-Service Ticket Creation vs. Booking Submit (`b09` vs `c09`)**
   - *Objective:* Test concurrency when a new maintenance ticket with initial **out-of-service** status is created concurrently with a booking submission.
   - *Baseline Result:* Raw DML inserts result in unhandled trigger exceptions (`Msg 50000`) when one transaction commits before the other.
   - *Controlled Result:* Maintenance ticket creation acquiring initial out-of-service impact acquires the Exclusive AppLock. The ticket creation completes (`rc = 0`), while the concurrent booking attempt is rejected cleanly with `51002`, guaranteeing $Q_{NR6} = 0$.
6. **Scenario 6: Dual Staff Approval Race (`b10` vs `c10`)**
   - *Objective:* Test the race condition where two staff members simultaneously approve two different pending requests for overlapping intervals on the same space.
   - *Baseline Result:* Under RCSI, both staff approval transactions read pending status from `tempdb` and insert approval rows without blocking, committing two overlapping approved bookings to disk ($Q_{BR1} = 1$).
   - *Controlled Result:* `usp_booking_approve` serializes all staff approvals per space. The first approval succeeds (`rc = 0`); the second approval rechecks confirmed bookings inside the lock, detects the first approval, and fails with `51003`, preserving $Q_{BR1} = 0$.
7. **Scenario 7: Soft-Gate Fallback for Disallowed Purposes (`c11`)**
   - *Objective:* Test business logic fallback when a user submits an instant booking with an ineligible purpose (e.g., `lecture` on a `meeting_room`).
   - *Controlled Result:* `usp_booking_instant_submit` validates space purpose eligibility. Instead of throwing an error or auto-approving, it inserts the request under `pending` status (`rc = 0`, `instant_accepted = 0`), routing it for manual staff review.
8. **Scenario 8: Pending Fallback vs. Instant Submit Race (`c12`)**
   - *Objective:* Ensure that a pending fallback booking does not block a subsequent instant booking for the same interval, but any later approval of the pending booking is correctly rejected.

- *Controlled Result:* The instant booking successfully acquires the space lock and auto-approves (`rc = 0`, `instant_accepted = 1`). A subsequent attempt by staff to approve the earlier pending fallback booking is rejected cleanly with `51003`.

9. **Scenario 9: Advisory Maintenance Acknowledgement Repair (`c13`)**

- *Objective:* Verify automatic repair and creation of advisory maintenance acknowledgement records during staff approval over spaces with active advisory notices.
- *Controlled Result:* `usp_booking_approve` detects an unacknowledged advisory maintenance notice on the space, approves the booking (`rc = 0`), and automatically inserts the missing row into `dbo.booking_advisory_acknowledgement`.

5.3.4 Comparison Results & Invariant Verification

Table 7 summarizes the comparative results across all test scenarios.

Test Scenario	**Concurrent Actions**	**Baseline (Raw DML)**	**Controlled (Task 12)**
K1 (`b01/c01`)	Two instant submissions for same space & time	Trigger bypassed under RCSI; overlapping bookings committed ($Q_{BR1} = 1$)	Serialized via AppLock; winner `rc=0`, loser `51003`, $Q_{BR1} = 0$
K2 (`b02/c02`)	Instant submit vs staff approval	Trigger bypassed under RCSI; overlapping booking committed ($Q_{BR1} = 1$)	Serialized; approval wins (`rc=0`), submit rejected (`51003`), $Q_{BR1} = 0$
K3 (`b03/c03`)	OOS escalation vs in-flight submit	Raw trigger `Msg 50000` exception or booking leak	Serialized; escalation wins (`rc=0`), submit/approve rejected (`51002`)
T5/T7 (`b05/c05`)	Lock timeout & retry contract	Uncontrolled blocking for 18+ seconds	First contender times out with `51005` (5s); retry succeeds (`rc=0`)
K5 (`b09/c09`)	OOS ticket creation vs instant submit	Raw trigger `Msg 50000` exception	Serialized; OOS ticket wins (`rc=0`), submit rejected (`51002`)
Staff Race (`b10/c10`)	Two staff members approving overlapping pendings	Trigger bypassed under RCSI; both approvals commit ($Q_{BR1} = 1$)	Serialized; first approval wins (`rc=0`), second rejected (`51003`)
Soft Gate (`c11`)	Instant submit with invalid purpose (`lecture`)	N/A	Falls back to pending (`rc=0`, `instant=0`) without auto-approval
Fallback Race (`c12`)	Pending fallback vs new instant submit	N/A	Instant wins (`rc=0`); later approval of pending rejected (`51003`)
Ack Repair (`c13`)	Staff approval under advisory maintenance	N/A	Approval succeeds (`rc=0`); auto-writes advisory ack record

Bång 7: Comparative concurrency test results between Baseline (Raw DML) and Controlled (Task 12 Procedures)

The suite-wide invariant audit script (`audit_invariant.sql`) executed at the conclusion of the test suite confirmed:

```
PASS suite-audit: zero overlapping confirmed pairs AND zero confirmed-vs-OOS overlaps on
    TEST-13 spaces.
```

This empirically proves that the Task 12 application-lock protocol guarantees 100% data integrity ($Q_{BR1} = 0, Q_{NR6} = 0$) under concurrent execution.

6 Sample Data Generation

This section describes `14-data-generator` and the generated Phase 2 dataset.

6.1 Scale and Time Coverage

Report the number of rows generated for each important table. Confirm that the dataset contains at least 100,000 bookings and spans three academic years.

Table or data category	Rows	Coverage notes
Bookings	0	Replace with the generated count and date range

Bång 8: Generated Phase 2 data volume

6.2 Generation Strategy

Explain distributions, reproducibility controls such as random seeds, referential integrity handling, and how realistic booking patterns were produced.

6.3 Exceptional Cases

Describe how cancellations, no-shows, advisory acknowledgements, maintenance overlaps, and other edge cases were deliberately seeded and validated.

7 Analytical Queries

Phase 2 (project description §1.3) introduces four reporting queries that support the Facility Manager with accumulated booking and maintenance history: (1) total approved booking hours per space for a semester, (2) the number of approved bookings by weekday and hour for a semester, (3) available spaces satisfying a required capacity and facility list within a time period, and (4) approved bookings affected when a maintenance record is escalated to out-of-service. This section describes their implementation in `outputs/16-analytical-queries-G05.sql` and the results observed on the generated dataset.

Each query is wrapped in a stored procedure (`dbo.usp_task16_q1` ... `dbo.usp_task16_q5`) so that the same query body can be executed repeatedly with different parameter sets, and each procedure validates its inputs (for example, it rejects a slot with `@slot_end <= @slot_start` with a descriptive `THROW`). The procedures share a small set of conventions derived from the Task 09 schema:

- *Confirmed bookings* are those with status `approved`, `checked_in`, or `completed`;
- soft-deleted rows (`is_deleted = 1`) are always excluded;
- time intervals are compared with the half-open overlap predicate `@start < row_end AND @end > row_start`;
- semester windows follow the U4 convention: Semester 1 is the half-open interval `[September 1, February 1)`, excluded summer.

All results in this section were obtained on 2026-08-08 by executing the complete script against the Task 14 scratch database `CS486_G05_T14` (120,000 bookings); the raw evidence is recorded in `logs/eval/task16/`. The semester examples use Semester 1 of academic year 2024–2025, `[2024-09-01, 2025-02-01)`.

Stored procedure	Purpose
`usp_task16_q3_total_approved_hours_per_space`	Report 1: approved booking hours per space
`usp_task16_q4_weekday_hour_demand`	Report 2: approved bookings by weekday and hour
`usp_task16_q2_room_finder`	Report 3: available spaces with capacity and required facilities
`usp_task16_q5_escalation_impact`	Report 4: approved bookings affected by escalation
`usp_task16_q1_booking_conflict_check`	Booking-conflict check; read hint for the Task 15 tuning workload

Bång 9: Analytical query inventory

The booking-conflict check is not one of the four reports: it shares its conflict semantics with the concurrency-safe write procedures of Task 12 and is analysed as the Task 15 tuning workload in Section 8. Reports 1–4 are detailed below.

7.1 Approved Booking Hours per Space (Report 1)

The first report answers: *how many confirmed booking hours did each space have in a semester?* All confirmed, non-deleted bookings whose requested interval overlaps the semester window are included, and each duration is clipped to the window so that a booking spilling over the semester boundary contributes only its in-semester hours. The aggregation groups by the stable space identity and orders by total hours descending so that the most loaded spaces appear first:

```
WITH semester_bookings AS (
    SELECT
        booking_id,
        space_id,
        CASE
            WHEN requested_start_time < @semester_start
                THEN @semester_start
            ELSE requested_start_time
        END AS eff_start,
        CASE
            WHEN requested_end_time > @semester_end
                THEN @semester_end
            ELSE requested_end_time
        END AS eff_end
    FROM dbo.bookings
    WHERE is_deleted = 0
```

```
      AND status IN ('approved', 'checked_in', 'completed')
      AND requested_start_time < @semester_end
      AND requested_end_time > @semester_start
)
SELECT
    s.space_id,
    s.space_code,
    s.space_name,
    s.capacity,
    COUNT(sb.booking_id) AS confirmed_booking_count,
    ROUND(
        SUM(DATEDIFF_BIG(SECOND, sb.eff_start, sb.eff_end) / 3600.0),
        2
    ) AS total_hours_in_semester,
    ROUND(
        SUM(DATEDIFF_BIG(SECOND, sb.eff_start, sb.eff_end) / 3600.0)
            / NULLIF(COUNT(sb.booking_id), 0),
        2
    ) AS avg_hours_per_booking
FROM dbo.spaces s
LEFT JOIN semester_bookings sb
    ON sb.space_id = s.space_id
GROUP BY
    s.space_id,
    s.space_code,
    s.space_name,
    s.capacity
ORDER BY
    total_hours_in_semester DESC,
    s.space_code ASC;
```

Executed with `@semester_start = '2024-09-01'` and `@semester_end = '2025-02-01'`, the query returned 120 spaces covering 901 confirmed booking intervals; Table 10 shows the top of the ordering.

Space	Cap.	Bookings	Hours	Avg hrs
Project Lab 131 (T14-BE-0032)	35	113	167.50	1.48
Project Lab 202 (T14-AL-0103)	40	111	164.00	1.48
Meeting Room 159 (T14-IN-0060)	30	100	147.50	1.48
Auditorium 200 (T14-LI-0101)	240	94	147.00	1.56
Computer Lab 165 (T14-IN-0066)	35	98	143.50	1.46
Classroom 117 (T14-IN-0018)	65	91	142.00	1.56

Bång 10: Top spaces by total approved booking hours, Semester 1 AY2024–2025

The aggregation key is the surrogate `space_id`, so the grouping is stable even though the same space name recurs across buildings (space identifiers 205031–205017 above are omitted from the table for readability). The duration is derived from the *requested* interval, per the U4 decision, rather than the actual session duration.

7.2 Bookings by Weekday and Hour (Report 2)

This query characterises the demand pattern of a semester: *on which weekdays and hours do approved bookings start?* Knowing the weekday–hour distribution of confirmed booking starts lets the Facility Manager align staffing and opening hours with real demand. The weekday number must be meaningful regardless of the session's `DATEFIRST`, so it is computed with a

pure integer expression anchored at `1900-01-01` and mapped with a modulo-7 arithmetic to `1 = Monday ... 7 = Sunday`:

```sql
WITH semester_bookings AS (
    SELECT
        booking_id,
        space_id,
        requested_start_time
    FROM dbo.bookings
    WHERE is_deleted = 0
      AND status IN ('approved', 'checked_in', 'completed')
      AND requested_start_time >= @semester_start
      AND requested_start_time < @semester_end
)
SELECT
    ((DATEDIFF(
        DAY,
        CONVERT(date, '19000101'),
        CONVERT(date, b.requested_start_time)
    ) % 7) + 7) % 7 + 1 AS weekday_number,
    CASE ((DATEDIFF(
        DAY,
        CONVERT(date, '19000101'),
        CONVERT(date, b.requested_start_time)
    ) % 7) + 7) % 7 + 1
        WHEN 1 THEN N'Monday'
        WHEN 2 THEN N'Tuesday'
        WHEN 3 THEN N'Wednesday'
        WHEN 4 THEN N'Thursday'
        WHEN 5 THEN N'Friday'
        WHEN 6 THEN N'Saturday'
        ELSE N'Sunday'
    END AS weekday_name,
    DATEPART(HOUR, b.requested_start_time) AS start_hour,
    COUNT(*) AS booking_count,
    COUNT(DISTINCT b.space_id) AS distinct_space_count
FROM dbo.bookings b
GROUP BY
    ((DATEDIFF(
        DAY,
        CONVERT(date, '19000101'),
        CONVERT(date, b.requested_start_time)
    ) % 7) + 7) % 7 + 1,
    DATEPART(HOUR, b.requested_start_time)
ORDER BY
    weekday_number,
    start_hour;
```

A half-open window is used: a booking contributes when its `requested_start_time` lies in `[@semester_start, @semester_end)`. With the same Semester 1 window the evaluation run produced 66 weekday/hour buckets, verified in the live evaluation; each bucket counts the confirmed bookings that start in that hour and the number of distinct spaces involved:

The row-level excerpt can be reproduced by re-running the procedure with the same window against the Task 14 database; the scheme of the result is stable, so the report can be regenerated for any semester without modifying the query body.

Column	Content
`weekday_number`, `weekday_name`	weekday of the booking start, Monday = 1
`start_hour`	hour of `requested_start_time` (0–23)
`booking_count`	number of confirmed bookings in the bucket
`distinct_space_count`	number of distinct spaces used in the bucket

Bång 11: Weekday/hour bucket schema of Report 2

7.3 Available Spaces with Required Facilities (Report 3)

This report implements a room finder: *which spaces satisfy a requested capacity and every required facility during a given time period?* The caller passes the slot, the minimum capacity, and the list of required facilities as a JSON array, which the procedure normalises into a local table variable before evaluating the availability criteria. Three conditions must hold simultaneously for a space to appear in the result:

1. *Capacity and facility fit* — `capacity >= @minimum_capacity` and the space has *every* requested facility, verified by relational division (a double `NOT EXISTS`); a one-facility match is not sufficient;
2. *No booking conflict* — no confirmed, non-deleted booking of that space overlaps the proposed slot;
3. *No hard maintenance conflict* — no `open` or `in_progress` maintenance with impact level `out-of-service` overlaps the slot; advisory maintenance does *not* block availability.

The core of the procedure is:

```
DECLARE @required_facilities TABLE (
    facility_name NVARCHAR(255) PRIMARY KEY
);

INSERT INTO @required_facilities (facility_name)
SELECT DISTINCT CAST([value] AS NVARCHAR(255))
FROM OPENJSON(@required_facilities_json)
WHERE [value] IS NOT NULL
  AND LTRIM(RTRIM(CAST([value] AS NVARCHAR(255)))) <> N'';

WITH booking_conflicts AS (
    SELECT DISTINCT b.space_id
    FROM dbo.bookings b
    WHERE b.is_deleted = 0
      AND b.status IN ('approved', 'checked_in', 'completed')
      AND @slot_start < b.requested_end_time
      AND @slot_end > b.requested_start_time
),
maintenance_conflicts AS (
    SELECT DISTINCT m.space_id
    FROM dbo.maintenance m
    WHERE m.is_deleted = 0
      AND m.status IN ('open', 'in_progress')
      AND m.impact_level = 'out-of-service'
      AND m.start_time < @slot_end
      AND (
          m.completion_time IS NULL
          OR m.completion_time > @slot_start
      )
),
```

```
facilities_ok AS (
    SELECT s2.space_id
    FROM dbo.spaces s2
    WHERE NOT EXISTS (
        SELECT 1
        FROM @required_facilities rf
        WHERE NOT EXISTS (
            SELECT 1
            FROM dbo.space_facilities sf
            INNER JOIN dbo.facilities f
                ON f.facility_id = sf.facility_id
            WHERE sf.space_id = s2.space_id
              AND f.name = rf.facility_name
        )
    )
)
SELECT
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    s.capacity,
    s.current_status AS status_hint,
    STRING_AGG(f.name, ', ') AS facility_summary
FROM dbo.spaces s
LEFT JOIN dbo.space_facilities sf
    ON sf.space_id = s.space_id
LEFT JOIN dbo.facilities f
    ON f.facility_id = sf.facility_id
LEFT JOIN booking_conflicts bc
    ON bc.space_id = s.space_id
LEFT JOIN maintenance_conflicts mc
    ON mc.space_id = s.space_id
WHERE s.capacity >= @minimum_capacity
  AND s.current_status NOT IN ('retired', 'temporarily_closed')
  AND bc.space_id IS NULL
  AND mc.space_id IS NULL
  AND EXISTS (
      SELECT 1
      FROM facilities_ok fo
      WHERE fo.space_id = s.space_id
  )
GROUP BY
    s.space_id,
    s.space_code,
    s.space_name,
    s.space_type,
    s.capacity,
    s.current_status
ORDER BY
    s.capacity ASC,
    s.space_code ASC;
```

The facility summary aggregates every facility of a qualifying space with `STRING_AGG`, so the caller sees exactly what is installed. Executed with `@slot_start = '2024-09-16T10:00:00'`, `@slot_end = '2024-09-16T12:00:00'`, `@minimum_capacity = 20` and the two-facility list `N'["T14 Projector","T14 Whiteboard"]'`, the query found exactly one available space (as expected from the multi-facility matching): Computer Lab 165 (`T14-IN-0066`, space 205065) with capacity 35, status `available`, and the facility summary shown in Table 12. With the

empty facility list the same query returns all capacities and locations free of booking and maintenance conflicts.

Property	Value
Space	`205065` — Computer Lab 165 (`T14-IN-0066`)
Type / capacity	`computer_lab` / 35
Status hint	`available`
Facility list	T14 Desktop Computer x3, T14 Livestreaming Kit x1, T14 Presenter Remote x4, T14 Projector x4, T14 Video Conference Unit x3, T14 Whiteboard x1
Requested window	2024-09-16 10:00–12:00

Bång 12: Room-finder result for a two-facility search

7.4 Approved Bookings Affected by Escalation (Report 4)

The last report supports business rule NR4. When the impact level of an *advisory* maintenance record is escalated to `out-of-service`, all already-approved bookings that overlap the maintenance period must be identified so that staff can contact the requesters. The query therefore operates on maintenance *history* events rather than on the current impact level of the maintenance record: it filters `maintenance_impact_history` for rows with `prior_level = 'advisory'` and `new_level = 'out-of-service'`, joins those events to the affected maintenance record, to the confirmed bookings overlapping the maintenance interval, and to the advisory acknowledgement recorded when the booking was submitted:

```
WITH escalation_events AS (
    SELECT
        mih.history_id,
        mih.maintenance_id,
        mih.changed_at AS escalation_time,
        mih.changed_by,
        mih.prior_level,
        mih.new_level,
        mih.reason AS escalation_reason,
        m.space_id,
        m.problem_description,
        m.start_time AS maintenance_start_time,
        m.completion_time AS maintenance_end_time
    FROM dbo.maintenance_impact_history mih
    INNER JOIN dbo.maintenance m
        ON m.maintenance_id = mih.maintenance_id
    WHERE mih.prior_level = 'advisory'
      AND mih.new_level = 'out-of-service'
      AND m.is_deleted = 0
      AND (
          @from_escalation_time IS NULL
          OR mih.changed_at >= @from_escalation_time
      )
      AND (
          @to_escalation_time IS NULL
          OR mih.changed_at < @to_escalation_time
      )
      AND (
          @maintenance_id IS NULL
```

```
            OR mih.maintenance_id = @maintenance_id
        )
)
SELECT
    ee.escalation_time,
    escalator.full_name AS escalated_by,
    ee.new_level,
    s.space_id,
    s.space_code,
    s.space_name,
    b.booking_id,
    b.requested_start_time,
    b.requested_end_time,
    b.purpose,
    b.status AS booking_status,
    requester.full_name AS requester_name,
    requester.email AS requester_email,
    baa.acknowledged_at,
    ROUND(overlap_seconds / 3600.0, 2) AS overlap_hours
FROM escalation_events ee
INNER JOIN dbo.booking_approvals ba
    ON ba.decision = 'approved'
   AND ba.decision_time <= ee.escalation_time
INNER JOIN dbo.bookings b
    ON b.booking_id = ba.booking_id
   AND b.space_id = ee.space_id
   AND b.is_deleted = 0
   AND b.status IN ('approved', 'checked_in', 'completed')
   AND b.requested_start_time < COALESCE(
       ee.maintenance_end_time,
       CONVERT(DATETIME2, '9999-12-31T23:59:59')
   )
   AND b.requested_end_time > ee.maintenance_start_time
INNER JOIN dbo.booking_advisory_acknowledgement baa
    ON baa.booking_id = b.booking_id
   AND baa.maintenance_id = ee.maintenance_id
INNER JOIN dbo.spaces s
    ON s.space_id = ee.space_id
INNER JOIN dbo.users requester
    ON requester.user_id = b.requester_id
INNER JOIN dbo.users escalator
    ON escalator.user_id = ee.changed_by
ORDER BY
    ee.escalation_time DESC,
    b.requested_start_time ASC,
    b.booking_id ASC;
```

The dependency on the acknowledgement join is deliberate: a booking can be reported only if the requester was informed of the advisory at submission time, which is exactly the population that NR4 wants contacted. For each row the procedure also computes `overlap_hours`, the portion of the booking interval that falls inside the maintenance period, so that staff can reprioritise the follow-up by impact size. The query is strictly read-only: it never mutates or cancels bookings (the concurrency-safe cancellation procedures of Task 12 are the only writers in that domain).

Executed with all-NULL filters (i.e., all escalation history), the live run returned 655 affected confirmed bookings. Table 13 shows the first rows of the result, referring to the escalation of maintenance 207478 (“Two ceiling lights flicker intermittently”, Meeting Room 110):

Each row additionally carries the requested time window, purpose, booking status, requester

Hist. id	Escalated at	By	Maint id	Booking	Requester
1	2026-08-07 16:55:56	Lan Hoang	207478	368665	Vinh Do
1	2026-08-07 16:55:56	Lan Hoang	207478	321574	Son Nguyen
1	2026-08-07 16:55:56	Lan Hoang	207478	386469	Trang Tran

Bảng 13: Escalation-affected bookings (excerpt of the 655 returned rows)

email and phone, the acknowledgement timestamp, and the computed `overlap_hours`; those columns are condensed in the excerpt above.

7.5 Work Assignment

The four reports were implemented by the group members listed below, each student contributing one query to the shared script:

Student	Report	Primary contribution
Phạm Hữu Nam (24125015)	Report 1	Semester-aggregated approved hours with the U4 window
Cao Quảng Hưng (24125010)	Report 2	Deterministic Monday=1 weekday grid
Trần Đình Quốc Thắng (24125079)	Report 3	JSON facility list with relational division
Nguyễn Hữu Phước (24125018)	Report 4	Escalation-history join with approval and acknowledgement

Bảng 14: Individual analytical-query contributions

8 Indexing & Query Tuning

This section summarizes `15-index-tuning-report` using measurements from the large Phase 2 dataset.

8.1 Selected Workloads

Evaluate the booking-conflict check, the room-finder query, and two selected analytical reports. State the parameters and data volume used so that the measurements are reproducible.

8.2 Indexes and Rationale

List each added or changed index, its key and included columns, and the query pattern it supports. Also discuss write and storage costs where relevant.

8.3 Before-and-After Results

Compare execution plans, elapsed time, CPU time, logical reads, or the metrics available in the selected database platform.

Query	Index change	Before	After	Plan change
To be completed	To be completed	0	0	To be completed

Bång 15: Query-tuning results on the generated dataset

9 Normalization Validation

9.1 Functional Dependencies

Identify the relevant functional dependencies for every new or changed Phase 2 relation. Distinguish candidate keys from non-key determinants.

Relation	Functional dependencies	Candidate keys
To be completed	To be completed	To be completed

Bång 16: Functional dependencies in Phase 2 relations

9.2 Third Normal Form Validation

For each relation, show that every non-trivial dependency has a superkey as its determinant or a prime attribute on its right-hand side. Record and correct any violation discovered during the review.

10 Conclusion

Phase 2 successfully extended the CS486 Campus Space Management System from a static Phase 1 relational baseline into a high-concurrency, enterprise-ready database architecture. By addressing requirement changes (Task 08), updating logical/conceptual models (Task 09), and executing non-destructive schema migrations (Task 10), the system expanded to support instant bookings, maintenance escalation workflows, soft-gate fallbacks, and advisory notices without breaking Phase 1 invariants.

10.1 Key Technical Accomplishments

- **Deterministic Concurrency Control:** The implementation of SQL Server Application Locks (`sys.sp_getapplock` on `space_booking:<space_id>`) resolved the critical read–check–write race conditions (K1–K5). Dual-session parallel test execution (Task 13) empirically verified 100% data invariant preservation ($Q_{BR1} = 0, Q_{NR6} = 0$) and eliminated unhandled database engine exceptions (`Msg 50000`) in client applications.
- **High-Volume Performance & Analytical Scaling:** The system demonstrated scalability under a synthetically generated dataset of over 100,000 bookings spanning three academic years (Task 14). Targeted indexing strategies (Task 15) optimized analytical reporting queries (Task 16), reducing query execution times by an order of magnitude.
- **Structured Agentic Engineering Methodology:** Operating within a strict 16-task design pipeline enforced rigorous traceability between business rules, normalized relational models (3NF/BCNF), and executable T-SQL DML/DDL artifacts.

10.2 Future Directions

While the current architecture satisfies all Phase 2 operational requirements, future production deployments could explore:

1. **Partition Pruning:** Implementing range partitioning on `bookings.requested_start_time` by academic semester to accelerate archival queries and partition maintenance.

2. **Per-Day Lock Granularity:** If production monitoring detects heavy lock contention on ultra-popular spaces, migrating from per-space lock keys to per-space-day keys (`space_booking:<space_id>:<date>`) offers a documented tuning path.